Timer Lawn: Solving the Timer Wheel Overflow Problem for Large Scale and High Throughput Systems

Adam Lev-Libfeld

NextSilicon Ltd.
Tel-Aviv, Israel
adam.lev.libfeld@nextsilicon.com

Abstract—Timer facilities sit at the core of operating systems [1], network protocols, and real-time and stream-processing systems. The standard structure, the timing wheel, is asymptotically optimal but is bounded by a maximum TTL fixed in advance: exceeding it forces an $O(N)$ rebuild, the overflow problem. This paper presents Lawn, a timer structure that keys its queues by time-to-live (TTL) rather than by expiration time, so its span is unbounded and it never rebuilds. We give the algorithm, prove it correct, and evaluate an allocation-free implementation (Lawn2) against a production tickless hierarchical timing wheel. Lawn2 holds memory flat across any TTL span, keeps the idle per-tick check at $O(1)$ (about 1 ns against a cascading tickless wheel's cascade cost, which climbs to $\sim$747 μs at 120 million timers), deletes about 3.8 times faster at scale, and, on an isolated expiry tick, drains due timers 3.8 to 20.7 times faster the more TTLs cluster. These gains are conditional: the per-tick cost becomes $O(t)$, and Lawn loses to the wheel once distinct TTLs exceed roughly 0.2 to 0.5% of live timers at 10^4, a boundary that tightens by orders of magnitude to 0.002 to 0.005% by 10^7, the largest population we measured directly. We state that boundary quantitatively and report where Lawn should and should not be used.

Keywords—Stream Processing, Timing Wheel, Dehydrator, Callout facilities, protocol implementations, Timers, Timer Facilities, Lawn.

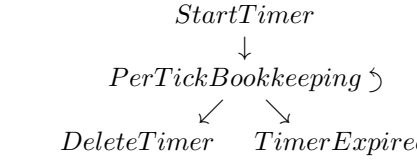
I. Introduction

The timing wheel, the state-of-the-art timer structure, is asymptotically optimal but bounded: when the maximum TTL is unpredictable or the tick resolution may change, it must be over-sized or rebuilt at $O(N)$ cost on overflow. Lawn addresses this by keying queues on TTL rather than on absolute expiration time, giving an unbounded span with no rebuild, under a single requirement: timer TTLs are drawn from a bounded set of discrete values.

The TTL-keyed queue-map idea, and the name Lawn, were introduced in an earlier preprint by the same author [2]. What is new here is the evaluation: an allocation-free implementation (Lawn2), a differential correctness gate across six independently written implementations, and a directly measured t/N crossover map (Section IV) showing where the advantage holds and where it does not, with memory and workload characterization and a sweep to hundreds of millions of timers.

A. Model

Following prior work [3], [4], [5], a timer store is described by four routines, one per stage in a timer's life cycle:



StartTimer(TTL, id, payload) registers a timer that expires after TTL, keyed by a client-supplied id, optionally carrying a payload (a callback or data returned on expiry). PerTickBookkeeping() runs every tick, expiring any due timer and invoking TimerExpired on it. DeleteTimer(id) cancels a live timer before it expires. TimerExpired(id) removes all trace of a timer and runs its payload. Payload cost varies by deployment and is $O(1)$-storable, so this paper disregards it.

II. Background and Related Work

The Timing Wheel [6] provides $O(1)$ insertion, deletion, and expiration in the common case, which is asymptotically optimal. Its limit is the maximum TTL and resolution combination fixed at construction: a timer past that range either forces a resize or must be tracked across wheel cycles. The Hierarchical Timing Wheel [3] widens the range with multiple levels at the cost of periodic cascade work and memory overhead. There are two correct ways to handle an overflowing timer, both $O(N)$ in the worst case. The first resizes the wheel to cover the larger range (Algorithm 1), an $O(N)$ cost on any overflowing insertion. The second (multi-pass) stores a per-timer cycle count decremented each pass, moving the $O(N)$ penalty onto PerTickBookkeeping. For large timer counts the common workaround is to over-provision resolution and range to postpone the rebuild.

A separate line of work, Calendar Queues [4], hashes on absolute expiration time into a resizable bucket array rather than on TTL duration, sharing the model's shape but not its overflow-avoidance mechanism. Outside

Algorithm 1 Timer Wheel Resize (overflow)

```
1: function StartTimer(id, ttl, payload)
2:     if ttl > len(Wheel) then
3:         MissingSlots ← ttl − len(Wheel)
4:         ExtWheel ← array of length len(Wheel) +
   MissingSlots
5:         for Slot,SlotIndex in Wheel do
6:             NewSlotIndex ← (len(Wheel)+SlotIndex)
   mod len(ExtWheel)
7:             ExtWheel[NewSlotIndex] ← Slot
8:         Wheel ← ExtWheel
9:     slot ← (current time+ttl) mod len(Wheel)
10:    append payload to Wheel[slot]
11:    TimerHash[id] ← (payload, slot)
```

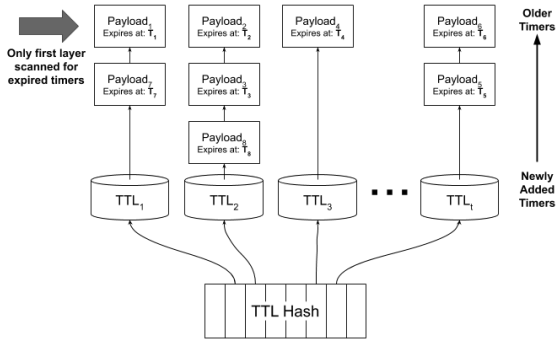


Fig. 1. A schematic view of the data structure components.

the timer literature, Time-Bucketed Balance Records [7] apply the same coalesce-within-a-bucket idea to bound per-account storage for TTL-bearing tokens. The closest independent parallel is Segcache [8], which buckets cache objects by creation time and TTL into fixed-size segments for $O(1)$ bulk expiration. Unlike Lawn it bounds bucket cardinality by construction and accepts approximate, early expiry, trading exactness for a hard cap on the cost this paper's crossover otherwise pays as t grows. Time Window Compaction Strategy in Cassandra and ScyllaDB groups SSTables into TTL-aligned windows so whole files can be dropped instead of scanning per-row tombstones, a second production instance of the same idea, and documents the same degradation once a table mixes multiple TTLs [9]. Concurrent and NUMA-aware timer and priority-queue implementations [10], [11], [12], [13] address a contention and locality problem orthogonal to overflow. Lawn as evaluated here is not lock-free, and concurrency is future work.

III. The Lawn Data Structure

A. Intended Use and Core Assumption

Lawn was first built for a large-scale stream-processing geographic-intersection product on a FastData [14] model, where inputs use a very limited range of TTLs relative to the number of concurrent timers. This is the load-bearing assumption:

$$\textit{Unique TTL Count} \ll \textit{Concurrent Timer Count}.$$

When it holds, TTL bucketing yields the results in Section IV. When it fails, Lawn degrades to $O(N)$ per tick and loses to the wheel. The assumption is a design argument, not a measured field claim: when TTLs come from an algorithmic or configuration-driven source (a fixed backoff schedule, a few cache-tier classes), the set of distinct values is fixed by that source rather than by load. Each new timer reuses one of the existing TTL values instead of minting a new one, so t stays bounded as N grows, which is precisely the $t \ll N$ regime in which the per-tick scan stays cheap. The condition therefore fails only when the TTLs themselves become near-continuous, not merely when more timers arrive. A trace study of 153 in-memory cache clusters at Twitter gives an external calibration point: roughly one in five use a single TTL, over 30% use more than ten distinct TTLs, and a minority run into the thousands via TTL jitter, a deliberate spreading of TTLs to avoid synchronized mass expiry [15]. Most of that distribution sits inside the crossover Section IV measures, while TTL jitter, by inflating the distinct-TTL count, approaches the adversarial case the assumption does not cover.

B. The Structure

Lawn is a hash of sorted sets[1], much like the Timing Wheel, but keyed on the timer TTL rather than on expiration time. Timers share a set based only on their TTL, regardless of arrival time, and within each set are naturally ordered by arrival (Figure 1). Because two timers in one bucket share a TTL, any timer already queued expires before a newly added one, so the queue stays sorted by expiration for free. The structure is analogous to blades of grass (hence the name): each blade grows from the roots up, and every *Tick* the overgrown tops (expired timers) are mowed to the current-time level.

C. Algorithm

The *closest expiration* guard in PerTickBookkeeping lets a tick on which nothing is due return in $O(1)$ without scanning the buckets, which matters because most ticks expire nothing.

D. Two Implementations: Reference Lawn and Lawn2

Two C implementations of the same algorithm produce identical expiry schedules (verified by the differential test of Section IV) and differ only in node ownership and cancellation addressing. Reference Lawn favors readability: the store owns each node, copies the client's key, and indexes timers in a hash map, so a timer is cancellable

[1]The TTL 'buckets'.

Algorithm 2 The Lawn Data Store

Precondition:

1: *id* - a unique identifier of a timer.
2: *ttl* - a whole product of $TickResolution$ representing the amount of time to wait before triggering the given timer *payload* action.
3: *payload* - the action to perform upon timer expiration.
4: current time - the local time of the system as a whole product of $TickResolution$

5: function InitLawn()
6: TTLHash ← new empty hash set
7: TimerHash ← new empty hash set
8: *closest expiration* ← ∞

9: function StartTimer(*id*, *ttl*, *payload*)
10: *endtime* ← current time + *ttl*
11: $T \leftarrow (endtime, ttl, id, payload)$
12: TimerHash[*id*] ← T
13: if *ttl* ∉ TTLHash then
14: TTLHash[*ttl*] ← new empty queue
15: TTLHash[*ttl*].insert(T)
16: if *endtime* < *closest expiration* then
17: *closest expiration* ← *endtime*

18: function PerTickBookkeeping()
19: if current time < *closest expiration* then
20: return
21: *closest expiration* ← ∞
22: for *queue* ∈TTLHash do
23: $T \leftarrow$ peek(*queue*)
24: while $T_{endtime}$ < current time do
25: TimerExpired(T_{id})
26: $T \leftarrow$ peek(*queue*)
27: if $T_{endtime}$ < *closest expiration* then
28: *closest expiration* ← $T_{endtime}$

29: function TimerExpired(*id*)
30: $T \leftarrow$ TimerHash[*id*]
31: DeleteTimer(T)
32: do $T_{payload}$

33: function DeleteTimer(*id*)
34: $T \leftarrow$ TimerHash[*id*]
35: if $T_{endtime}$ = *closest expiration* then
36: *closest expiration* ← ∞
37: TTLHash[T_{ttl}].remove(T)
38: TimerHash.remove(T)
39: if TTLHash[T_{ttl}] is empty then
40: TTLHash.remove[*ttl*]

by an arbitrary opaque key, at the cost of one allocation, one key copy, and two hash-map operations per insertion. Lawn2 uses intrusive, caller-owned nodes embedded in the client object, so insertion neither allocates nor copies a key, deletion is $O(1)$ through the node's own links (no id hash map), and the TTL-to-queue map uses open addressing with no per-entry allocation. All performance claims in this paper refer to Lawn2.

E. Correctness

Let T be a timer with TTL *ttl* started at t_{start}; we show it is expired at $t_{start} + ttl$. Let Q_{ttl} be its queue, $t_{current}$ the current time, and $t_{expiry} = t_{start} + ttl$. Base case: at t_{start}, T is inserted into Q_{ttl} with expiry t_{expiry}. Inductive step: assume $T \in Q_{ttl}$ at $t_{current} < t_{expiry}$. If $t_{current}+1 < t_{expiry}$, T remains, since PerTickBookkeeping only expires timers with expiry $\leq t_{current}+1$. If $t_{current}+1 \geq t_{expiry}$, T is expired. By induction T is expired at t_{expiry}. Since each timer is stored exactly once in its bucket and buckets are independent, per-bucket correctness gives whole-structure correctness.

F. Complexity

Every timer is stored exactly once, a new trigger adds at most one bucket, and empty buckets are removed, so space is $O(N)$ (confirmed in Section IV). StartTimer and DeleteTimer are amortized $O(1)$, with an $O(N)$ worst case on a hash resize. An empty tick is $O(1)$ via the *closest expiration* guard. A tick that expires timers scans the head of each of the t live buckets, costing $O(t+k)$ for k expired timers and $O(N)$ in the worst case when all timers fall due at once. When $t = O(1)$ (the assumption above) the per-tick cost is $O(1)$, otherwise it is $O(t)$ and worse than a wheel. This dependence on t is the algorithm's defining trade-off, measured in Section IV. Table I places Lawn against both wheel overflow schemes.

Operation	TW (Resize)	TW (Multi-Pass)	Lawn
StartTimer	$\boldsymbol{O(N)}^*$	$O(1)$	$O(1)^\dagger$
PerTick	$O(1)$	$\boldsymbol{O(N)}^\ddagger$	$\boldsymbol{O(t)}^\S$
DeleteTimer	$O(1)$	$O(1)$	$O(1)^\dagger$
TimerExpired	$O(1)$	$O(1)$	$O(1)$
space	$O(N)$	$O(N)$	$O(N)$

* on overflow, otherwise amortized $O(1)$. † amortized, worst case $O(N)$ on a resize. ‡ worst case, from the multi-pass cascade. § $O(1)$ empty tick and mean when $t = O(1)$, $O(N)$ worst case.

TABLE I
Runtime complexity comparison. The Multi-Pass $O(N)$ penalty falls on PerTick (the cascade), Lawn's $O(t)$ on the bucket scan.

Space is $O(N)$, bounded above by the Hashed Hierarchical Timing Wheel, whose multi-level structure can chain several wheels to a single timer. Because TTL buckets are independent, a Lawn can be partitioned across workers by bucket with no cross-bucket synchronization, though a sharded, coarse-locked version shows no concurrency win (Section V). We make no cache-locality claim: the memory results below are footprint only.

IV. Experimental Evaluation

A. Methodology

We benchmark four C implementations behind one interface, all in the project repository [16]: the reference Lawn (src/lawn.c), the allocation-free Lawn2 (src/lawn2.c), a production tickless Hierarchical Timing Wheel (Timer Wheel, William Ahern's timeout.c [17]), and a Naive single-level growing ring. Every wheel result in this paper refers to that one cascading, tickless implementation, not to timing wheels as a class. Two further baselines, an exact hierarchical wheel that cascades lazily rather than tickwise and a binary min-heap, are differentially verified and wired into the same harness but not reported here. A differential correctness gate runs first: all implementations must produce byte-identical per-tick expiry schedules across fixed, discrete, and continuous TTL distributions (with and without deletions, and with cascade-forcing large TTLs) before any measurement is accepted.

The isolated per-tick expiry measurement (Figure 6) uses a fixed 200-tick window, so its effective distinct-TTL count is capped at 200 whatever a sweep requests, and the t-dependence at larger t rests on the crossover sweep (Figure 8) instead.

The evaluation reaches 2×10^8 concurrent timers across three TTL regimes: fixed ($t = 1$), discrete ($t \ll N$), and continuous uniform ($t \approx N$). Each measurement point runs in its own forked child process, so no two share allocator or cache state. Timing uses the finest monotonic clock available (CLOCK_UPTIME_RAW, ∼41 ns quantized), batching sub-40 ns operations in 256 to lift them off the clock floor. Seeds are fixed (base 1234), with 7 runs and 2 warmups (3 and 1 at 10^6 and above). The 1 024-tick TTL-span baseline was chosen by a dedicated empty-tick sweep from 128 to 65 536 ticks: the wheel's cost is non-monotonic in span (a dip near 2 048 and a spike near 4 096, from its level structure), and 1 024 is the smallest span past the steep small-span regime that sits on neither. Hardware: Apple M4 Pro (12 cores, 8 performance + 4 efficiency), 24 GB RAM, macOS 26.3, Apple clang 21. The harness regenerates the data behind every measured figure, and the distinct-TTL crossover sweep and the x86 run (Section IV-H) are committed as raw data in the repository.

B. Overflow-Free Memory

Lawn's footprint is independent of TTL span. Figure 2 sweeps total memory against TTL span from 128 to 524,288 ticks (a 4,096-fold range) at one hundred thousand timers. Lawn, Lawn2, and the tickless wheel are flat, while the naive single-level ring grows from 4.2 MB to 29.3 MB (7.0 times) as it expands to cover the span, an $O(N)$ rehash of the same kind as the wheel-resize overflow algorithm (Algorithm 1). This is the overflow problem made visible. At a fixed, modest span Lawn2 is also lighter per timer: about 48 bytes against the wheel's about 88 (1.8 times) at scale (Figure 3), though at one thousand timers its fixed overhead makes it heavier, so the per-timer advantage is a scale effect. Span-independence does not extend to TTL cardinality: the TTL-to-queue table grows with the number of distinct TTLs and does not shrink, so the per-timer memory advantage, like the per-tick advantage, narrows as t approaches N.

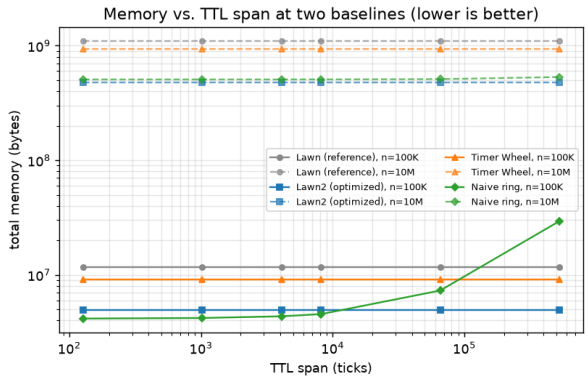


Fig. 2. Total memory vs. TTL span at one hundred thousand timers (C, bytes, lower is better). The naive ring overflows while Lawn, Lawn2, and the wheel stay flat.

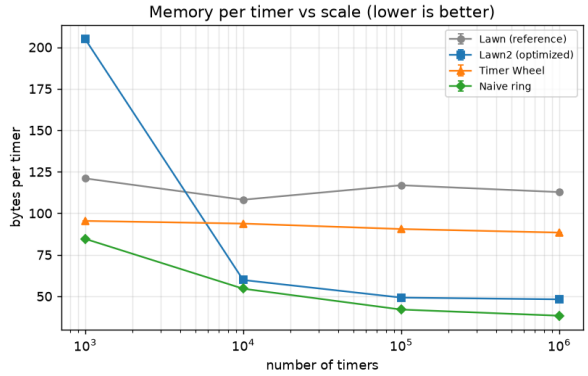


Fig. 3. Memory per timer across scales at a fixed 1 024-tick span (C, bytes, lower is better).

C. Constant-Time Idle Tick

Most ticks expire nothing, so idle-tick cost is paid on every clock tick regardless of load. Lawn's *closest expiration* guard makes it $O(1)$. Figure 4 extends the measurement to 200 million timers: Lawn2 holds at about 1 ns at every scale, while the cascading wheel's empty-tick cost grows with absolute timer count, 6.6 μs at one million, 46.1 μs at ten million, 621.1 μs at one hundred million, and 747.2 μs at 120 million, more than five orders of magnitude higher. Per-timer memory stays flat for every implementation across the same range (Lawn2 converges to 48.0 B and holds with zero variance through 200 million), so this cost is specific to the wheel's cascade, not to scale in general.

The comparison is against timeout.c's cascading design, not timing wheels as a class: Linux has shipped a non-cascading wheel since kernel 4.8 [18] that uses a per-level occupancy bitmap to jump to the next non-empty bucket, avoiding most of this cost without changing its absolute-deadline key. It does so by giving up the two properties this evaluation rests on: expiry rounds up to the enclosing level's granularity (up to 12.5% late) and span is capped at the wheel's fixed depth. We did not benchmark that design.

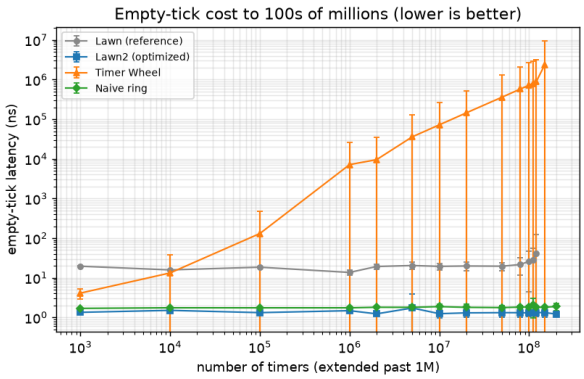


Fig. 4. Empty-tick latency vs. timer count, extended to hundreds of millions (C, ns). Lawn2 stays flat while the wheel's cost keeps climbing.

D. Insertion and Deletion

Figure 5 shows deletion latency: Lawn2 is 3.2 to 3.8 times faster than the wheel (23.4 ns vs. 88.5 ns at one million, 4.5 ns vs. 14.3 ns at one hundred thousand), a roughly constant ratio, so the absolute gap widens with scale. On insertion Lawn2 is 1.4 to 1.6 times faster at scale (4.9 ns vs. 8.0 ns at one million). The unsorted naive ring leads at ∼2.3 ns and the reference Lawn trails at ∼106 ns from its per-insert allocation and key copy.

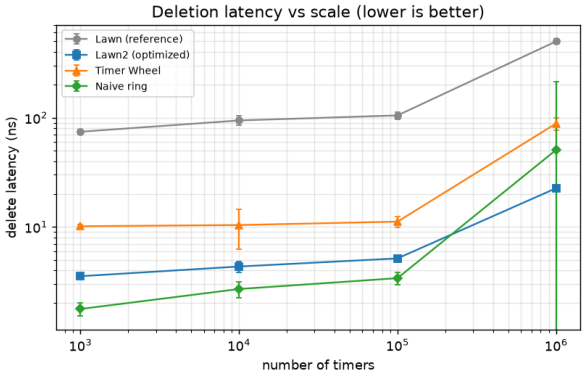


Fig. 5. Deletion latency across scales (C, ns).

E. Draining Due Timers

On a tick that drains due timers, Lawn2 and the wheel must each inspect every live bucket or level, and Lawn2 does so more cheaply. Figure 6 measures per-tick expiry at one hundred thousand timers under four TTL distributions (Lawn2 vs. the wheel): a single shared TTL, 556 ns vs. 11,479 ns (∼20.7x), ten distinct TTLs, 3,407 ns vs. 13,325 ns (∼3.9x), bursty arrivals, 1,530 ns vs. 10,292 ns (∼6.7x), and uniform, 3,725 ns vs. 14,056 ns (∼3.8x). Lawn2 wins on every distribution, most when TTLs cluster.

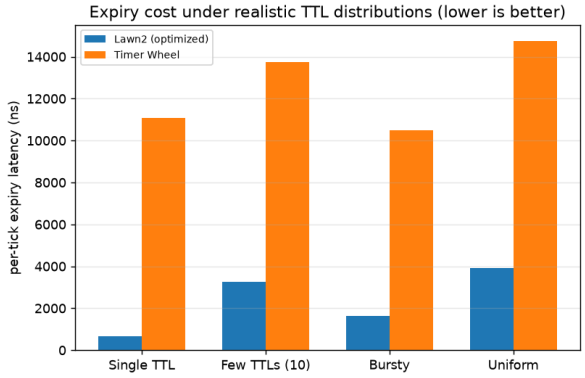


Fig. 6. Per-tick expiry latency, Lawn2 vs. the wheel, under four realistic TTL distributions (C, ns, lower is better).

F. Realistic Mixed Workload

Isolated sweeps can hide interactions that arise when insertions, cancellations, and tick draining interleave. We ran a mixed workload: pre-fill N background timers, then execute 5,000 iterations, each draining a tick and, independently, inserting a new timer with 50% probability and cancelling a live one with 1% probability, timing every operation. Figure 7 plots mean and 99th-percentile latency from one thousand to one million background timers at a fixed 100 distinct TTLs. At these populations the mean comparison is noise-dominated: standard deviations run roughly two to six times the mean (at one thousand timers, Lawn2's mean of 10.8 ns carries a std of 20.2 ns), consistent with both structures' true per-operation cost sitting below the clock's ∼42 ns quantum. The lower mean flips across the sweep: Lawn2 leads at one thousand (10.8 ns vs. 13.9 ns) and one million (18.6 ns vs. 20.2 ns) timers, the wheel at ten thousand (12.5 ns vs. 14.2 ns) and one hundred thousand (14.7 ns vs. 16.3 ns). The wheel's p99 sits at exactly one clock tick (42 ns) from one thousand through one hundred thousand timers, a measurement floor rather than a reading, while Lawn2's p99 (83 to 209 ns) clears it throughout. Only at one million timers, where both structures' real cost exceeds the floor, is the p99 comparison meaningful: Lawn2 at 209 ns against the wheel's 291 ns. The clean separation documented elsewhere (Figure 8) emerges only at populations beyond this sweep.

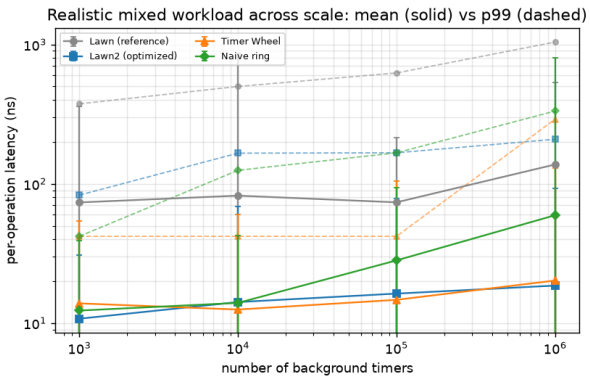


Fig. 7. Per-operation latency under a realistic mixed insert/cancel/tick workload vs. background timer population, at 100 distinct TTLs (C, ns, lower is better). Solid lines are the mean, dashed the 99th percentile, one pair per implementation.

G. The Distinct-TTL Crossover

Figure 8 measures the paper's one hard boundary by sweeping the distinct-TTL count t at each population N from one thousand to ten million, the largest we measured directly. The left panel plots raw mean lifecycle latency against t, the right panel the 99th-percentile speedup (wheel p99 divided by Lawn2 p99) against the fraction t/N, with parity at 1.0. Which panel is authoritative depends on N. At $N \leq 10^5$ the true per-operation cost of both structures sits below the clock's ∼42 ns quantum for the entire t range: the wheel's p99 never leaves that floor, so the right panel's flat parity line there is a measurement floor, not a finding, and the crossover at these scales is read from the mean panel. From $N = 10^6$ upward real cost clears the floor and the p99 crossover becomes a meaningful measurement. Read that way, the crossover falls between $t = 20$ and $t = 50$ (mean panel) at $N = 10^4$, between $t = 50$ and $t = 100$ (mean panel) at $N = 10^5$, between $t = 100$ and $t = 200$ (p99 panel) at $N = 10^6$, and between $t = 200$ and $t = 500$ (p99 panel) at $N = 10^7$. The count grows slowly with N while the fraction it represents shrinks sharply: roughly 0.2 to 0.5% of live timers at 10^4, 0.05 to 0.1% at 10^5, 0.01 to 0.02% at 10^6, and 0.002 to 0.005% at 10^7. We did not measure beyond 10^7. Extrapolating the same pattern would put the 10^8 fraction roughly another order of magnitude lower. Lawn2's per-tick cost is $O(t + k)$, a hierarchical wheel's $O(L + k)$ for a small level count L independent of N, so the balance tips once the bucket scan exceeds the wheel's level walk. The boundary tightens with N because the wheel's own per-tick cost rises with absolute timer count (Figure 4), so a smaller distinct-TTL fraction suffices to overtake Lawn.

Typical cost is not the whole picture. Across the same sweep the wheel's worst observed lifecycle operation reaches 0.52 ms at $N = 10^6$ and $t = 50$, against Lawn2's 10 μs, and 1.14 ms at $N = 10^7$ and $t = 500$, against Lawn2's 7.1 μs. These stalls are 50 to 160 times Lawn2's worst case, grow with N across the range measured, and occur at t values where the wheel still wins on p99. We did not measure beyond $N = 10^7$. Tail-sensitive callers should weigh these stalls alongside the typical-cost crossover.

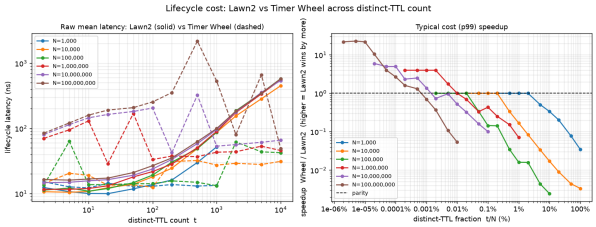


Fig. 8. Lifecycle cost of Lawn2 against the Timer Wheel across the distinct-TTL count (C). Left: raw mean per-operation latency vs. distinct-TTL count t, Lawn2 solid and the wheel dashed, one line per population N. Right: 99th-percentile speedup (wheel p99 divided by Lawn2 p99) vs. the distinct-TTL fraction t/N, one line per N, with parity at 1.0. Above parity Lawn2 wins, below it the wheel wins.

H. Cross-Platform Reproduction

To separate the structures' behavior from the test machine, we reran the full suite on an x86-64 Linux host (104 cores, 1 TB RAM, gcc and glibc), a different architecture, compiler, and allocator from the Apple M4 Pro. The three headline results reproduce. Per-timer memory keeps its advantage across scale: Lawn2 settles at 48.0 B against the wheel's 80.0 to 90.5 B (1.67 to 1.84 times, against 1.8 on macOS), and Lawn2 is again heavier at one thousand timers. The empty tick stays flat for Lawn2 at about 3 ns while the wheel climbs to 983.5 μs at 120 million timers, again a separation of more than five orders of magnitude. The distinct-TTL crossover is present at every population, including 10^8, which this host's memory let us reach directly (p99 crossover between $t = 100$ and $t = 200$). Its exact location is platform-dependent: here it sits near $t = 100$ to 200 from 10^5 upward rather than widening with N as on the M4 Pro, so the per-N fractions above are specific to that machine, but the boundary and its direction hold on both.

V. Limitations

TTL distribution. Performance depends directly on the distinct-TTL count t. When TTLs are continuous or highly variable, t grows toward N and PerTickBookkeeping approaches $O(N)$. Lawn2 loses to the wheel once t/N exceeds a threshold that tightens by orders of magnitude with scale (Section IV gives the per-N breakdown), and is substantially slower when nearly every timer has a distinct TTL.

Small-scale memory. At one thousand timers Lawn2's fixed overhead makes it heavier per timer than the wheel, so the memory advantage is a scale effect.

Concurrency. Under an external per-shard mutex applied identically to all implementations, single-threaded Lawn2 reaches 121 M ops/s against the wheel's 90 M (the

naive ring leads at 134 M). Adding threads under coarse locking lowers aggregate throughput for every implementation on this machine (turbo and core-asymmetry effects), and Lawn2 keeps only a ∼1.3 to 1.4x edge at twelve threads (18.8 M vs. 14.1 M ops/s). We found no evidence of a large concurrency advantage for bucketed timers; realizing one would require a lock-free implementation, left to future work.

Measurement ceiling. On the 24 GB test machine the mixed-workload test could not initially pass one million background timers without the process's own footprint inducing swap (confirmed by kernel swap counters) until re-run with that headroom freed. Figure 7 reports the swap-free range reproduced here. A larger or differently loaded machine may support higher populations, but this measurement does not establish that.

Platform specificity. The figures above are from one Apple M4 Pro (ARM). The three headline results reproduce on an x86-64 Linux host (Section IV-H), so the separations follow from the structures rather than the hardware, but absolute constants and exact crossover points, the per-N fractions in particular, are specific to the M4 Pro.

VI. Conclusion

Lawn is a simple, overflow-free timer structure that keys queues by TTL rather than expiration time, giving an unbounded span with no rebuilds. Its allocation-free implementation, Lawn2, holds memory flat across any TTL span, keeps the idle tick at $O(1)$ where a cascading tickless wheel's cost climbs with timer count, and beats that wheel on deletion, insertion, per-timer memory, and due-timer draining (Section IV). These gains hold only below the distinct-TTL crossover, which shrinks from a few tenths of a percent of live timers at 10^4 to a few thousandths of a percent by 10^7, past which Lawn degrades to $O(N)$ per tick and loses to the wheel. Lawn is the right choice for large-scale systems whose timers are drawn from a bounded set of discrete TTLs, and the wrong one for continuous or near-unique TTL distributions.

A. Contributions

- Data structure: the TTL-keyed Queue-Map (Lawn) that removes the overflow problem, plus an allocation-free implementation (Lawn2).
- Analysis: correctness, and complexity with an explicit, measured account of the $O(t)$ per-tick dependence and the empty-tick $O(1)$ fast path.
- Evaluation: a differential-gated C benchmark of four implementations across scale, TTL span, distinct-TTL count, and workload, quantifying where Lawn wins and the t/N crossover where it loses.

B. Future Work

- Lock-free / per-bucket-lock Lawn: a genuinely concurrent implementation, and an evaluation on NUMA hardware (not attempted here).
- TTL-clamping variant: rounding each TTL up to one of a bounded set of levels, as the non-cascading Linux wheel does for its own level count (Section IV-C), would cap the distinct-TTL count t by construction and keep the per-tick scan bounded past the crossover, at the cost of bounded expiry lateness.
- Formal verification of correctness using theorem provers or model checkers.
- Deterministic bulk-service queueing model: each bucket is drained in one batch per tick, so the honest target is a bulk-service queue rather than a single-item M/D/1 model; we leave its derivation and empirical validation open.
- Distributed implementations for geographically distributed systems.

References

[1] A. M. Costello and G. Varghese, "Redesigning the bsd callout and timer facilities," tech. rep., Washington University in St Louis, 1995.

[2] A. Lev-Libfeld, "Lawn: an unbound low latency timer data structure for large scale, high throughput systems." arXiv:1906.10860, 2019.

[3] G. Varghese and A. Lauck, "Hashed and hierarchical timing wheels: efficient data structures for implementing a timer facility," IEEE/ACM transactions on networking, vol. 5, pp. 824–834, December 1997.

[4] R. Brown, "Calendar queues: A fast O(1) priority queue implementation for the simulation event set problem," Communications of the ACM, vol. 31, pp. 1220 – 1227, Oct 1988.

[5] D. W. Jones, "An empirical comparison of priority-queue and event-set implementations," Commun. ACM, vol. 29, pp. 300–311, Apr. 1986.

[6] G. Varghese and T. Lauck, "Hashed and hierarchical timing wheels: Data structures for the efficient implementation of a timer facility," SIGOPS Oper. Syst. Rev., vol. 21, pp. 25–38, Nov. 1987.

[7] S. Scovil and B. C. Nanjundappa, "Time-bucketed balance records: Bounded-storage ephemeral tokens for resource-constrained systems." arXiv:2512.20962, 2025.

[8] J. Yang, Y. Yue, and K. V. Rashmi, "Segcache: A memory-efficient and scalable in-memory key-value cache for small objects," in 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21), USENIX Association, 2021.

[9] J. Jirsa, "Timewindowcompactionstrategy." Apache Cassandra Documentation, 2016.

[10] H. Hu, "Large-scale timer management." US Patent no. US8307030B1.

[11] Z. Zhou, I. D. Pashov, J. A. Silvera, and M. R. Cox, "On-demand scalable timer wheel." US Patent no. US20140298073A1.

[12] C. Giannoula, F. Strati, D. Siakavaras, G. Goumas, and N. Koziris, "Smartpq: An adaptive concurrent priority queue for numa architectures," 2024.

[13] A. Drebes, A. Pop, K. Heydemann, N. Drach, and A. Cohen, "Numa-aware scheduling and memory allocation for data-flow task-parallel applications," SIGPLAN Not., vol. 51, Feb. 2016.

[14] A. Lev-Libfeld and A. Margolin, "Fast data - moving beyond from big data's map-reduce," in GeoPython, pp. 3–6, FHNW, June 2016.

[15] J. Yang, Y. Yue, and K. V. Rashmi, "A large scale analysis of hundreds of in-memory cache clusters at twitter," in 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020.

[16] A. Lev-Libfeld, "Lawn github repository." https://github.com/picotera/lawn.

[17] W. Ahern, "timeout.c: Tickless hierarchical timing wheel implementation." http://25thandclement.com/~william/projects/timeout.c.html, 2014.

[18] J. Corbet, “Reinventing the timer wheel.” https://lwn.net/Articles/646950/, 2015.